

Guía de Estudio y Resumen: Desarrollo Web (UADE)

Este documento es una guía interactiva y detallada diseñada para ayudarte a repasar los conceptos fundamentales de la materia, enfocándose en CSS avanzado (posicionamiento, Flexbox, responsive) y la lógica de programación en JavaScript (DOM, formularios, modales, carruseles y algoritmos básicos).

1. Posicionamiento en CSS (**position**)

El posicionamiento determina cómo se colocan los elementos en el documento web y cómo interactúan entre sí dentro del flujo de la página.

static vs **relative** vs **absolute**

Valor de position	¿Sigue el flujo normal?	¿Responde a top , bottom , left , right ?	¿Respecto a qué se posiciona?	Caso de Uso Común
static	Sí (es el valor por defecto).	No.	Al flujo natural del documento.	Layouts normales.
relative	Sí (deja un "hueco" vacío donde habría estado).	Sí (desplaza el elemento sin alterar a los vecinos).	A su propia posición original .	Para desplazar ligeramente un elemento o actuar como contenedor de referencia para hijos absolutos.
absolute	No (sale por completo del flujo).	Sí (se ubica con precisión milimétrica).	Al ancestro posicionado más cercano (relative , absolute o fixed). Si no hay, a <body> .	Badges de notificación, menús desplegables, elementos decorativos superpuestos.

[!IMPORTANT] El Truco del Contenedor Relative: Si quieres posicionar un botón o elemento en la esquina exacta de una tarjeta (Card), debes ponerle **position: relative;** a la tarjeta (padre) y **position: absolute;** al botón (hijo). De lo contrario, el botón absoluto se posicionará respecto a la pantalla completa.

```
/* Ejemplo Práctico */
.card-padre {
  position: relative; /* Contenedor de referencia */
  width: 300px;
  height: 200px;
  border: 1px solid #ccc;
}

.badge-hijo {
```

```
position: absolute;
top: 10px;
right: 10px; /* Se pega arriba a la derecha de .card-padre */
background-color: red;
color: white;
}
```

fixed vs sticky

Característica	position: fixed	position: sticky
Relación con el Scroll	Siempre se queda inmóvil en el mismo lugar de la ventana gráfica (viewport), sin importar cuánto scroll se haga.	Se comporta como relative hasta que el scroll llega a un umbral definido (ej. top: 0), momento en el que se "pega" como fixed .
Flujo del Documento	Sale por completo del flujo. Los elementos contiguos suben para ocupar su espacio.	Mantiene su espacio en el flujo original. No altera los elementos circundantes al activarse.
Límite de Acción	Su contenedor es toda la ventana (pantalla).	Está limitado por los límites físicos de su contenedor padre . Si el padre sale de la pantalla, el elemento sticky también.
Caso de uso típico	Botón flotante de WhatsApp, barra de navegación superior persistente.	Cabeceras de tablas (<th>), barras de navegación de categorías internas.

```
/* Barra de navegación superior fija pase lo que pase */
.nav-fixed {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  z-index: 1000;
}

/* Título de sección que se queda arriba al scollar, pero solo dentro de su contenedor */
.section-header-sticky {
  position: sticky;
  top: 0;
  background-color: white;
  z-index: 10;
}
```

2. Flexbox y la Lógica de Ejes

Flexbox es un modelo de diseño unidimensional (organiza elementos en filas o columnas). Para entender Flexbox, debes dominar el concepto de **Ejes**.

```
graph LR
  subgraph Ejes_en_flex-direction_row [Ejes en flex-direction: row (Por Defecto)]
    R1[Eje Principal / Main Axis - Horizontal]
    R2[Eje Cruzado / Cross Axis - Vertical]
  end
  style R1 fill:#d4edda,stroke:#28a745,stroke-width:2px
  style R2 fill:#f8d7da,stroke:#dc3545,stroke-width:2px
```

```
graph TD
  subgraph Ejes_en_flex-direction_column [Ejes en flex-direction: column]
    C1[Eje Principal / Main Axis - Vertical]
    C2[Eje Cruzado / Cross Axis - Horizontal]
  end
  style C1 fill:#d4edda,stroke:#28a745,stroke-width:2px
  style C2 fill:#f8d7da,stroke:#dc3545,stroke-width:2px
```

Propiedades clave (El "Efecto Ranitas")

1. **flex-direction**: Define la dirección del Eje Principal.
 - **row** (defecto): Elementos horizontalmente de izquierda a derecha.
 - **column**: Elementos verticalmente de arriba a abajo.
2. **justify-content**: Alinea los elementos a lo largo del **Eje Principal**.
 - **flex-start**: Al principio.
 - **flex-end**: Al final.
 - **center**: Centrados.
 - **space-between**: Espaciados uniformemente (el primero al inicio, el último al final).
 - **space-around**: Espaciados con espacio equivalente a los lados.
 - **space-evenly**: Mismo espacio físico entre cualquier par de elementos y los bordes.
3. **align-items**: Alinea los elementos a lo largo del **Eje Cruzado**.
 - **stretch** (defecto): Estira los items para llenar el contenedor.
 - **flex-start**: Alineados al techo (arriba en row).
 - **flex-end**: Alineados al piso (abajo en row).
 - **center**: Centrados verticalmente.
 - **baseline**: Alineados según la línea base del texto.
4. **align-self**: Permite a un **único elemento** hijo invalidar la propiedad **align-items** del padre.
5. **flex-wrap**: Permite que los elementos salten de línea si no caben.
 - **nowrap** (defecto): Todo en una sola línea (se achican si es necesario).
 - **wrap**: Saltan a la siguiente línea si no hay espacio.

3. Media Queries y Flexbox Responsivo

Las Media Queries permiten aplicar estilos CSS específicos dependiendo de las características del dispositivo (principalmente el ancho de pantalla).

Ejemplo del Footer que pasa a vertical en Mobile

Es muy común que en computadoras (Desktop) un footer tenga varias columnas lado a lado (Flex horizontal), pero en celulares (Mobile) queremos que esas columnas se apilen verticalmente (Flex vertical) para que sea legible.

HTML

```
<footer class="main-footer">
  <div class="footer-col"><h3>Sobre Nosotros</h3><p>Info...</p></div>
  <div class="footer-col"><h3>Enlaces</h3><p>Contacto...</p></div>
  <div class="footer-col"><h3>Redes Sociales</h3><p>Instagram...</p></div>
</footer>
```

CSS (Enfoque Mobile-First)

El enfoque **Mobile-First** consiste en definir los estilos base para teléfonos móviles y luego usar Media Queries para añadir complejidad a pantallas más grandes.

```
/* Estilos Base: Pantallas Pequeñas (Mobile) */
.main-footer {
  display: flex;
  flex-direction: column; /* Apilados verticalmente */
  gap: 20px;
  padding: 20px;
  background-color: #1a1a1a;
  color: white;
}

.footer-col {
  width: 100%; /* Ocupa todo el ancho en mobile */
}

/* Media Query: Pantallas Medianas/Grandes (Desktop) */
@media (min-width: 768px) {
  .main-footer {
    flex-direction: row; /* Se ponen uno al lado del otro */
    justify-content: space-between; /* Distribuidos */
  }

  .footer-col {
    width: 30%; /* Cada columna ocupa casi un tercio */
  }
}
```

[!TIP] Al usar `min-width`, el navegador aplica las reglas de adentro del `@media` solo si la pantalla tiene **como mínimo** ese ancho (ej. 768px o más). Esto hace que tu CSS sea limpio y eficiente.

4. El Elemento `<picture>` para Diseño Adaptable

La etiqueta `<picture>` es un contenedor HTML que permite definir múltiples fuentes de imagen para que el navegador elija de forma inteligente la más adecuada según el tamaño de pantalla o soporte de formatos.

Estructura y Funcionamiento

Contiene una o más etiquetas `<source>` y **siempre** termina con una etiqueta `<img>` obligatoria (que sirve de fallback o respaldo si el navegador no entiende `<picture>`).

```
<picture>
  <!-- Si la pantalla tiene un ancho mínimo de 1024px, usa esta imagen grande -->
  <source media="(min-width: 1024px)" srcset="banner-desktop.jpg">

  <!-- Si la pantalla tiene un ancho mínimo de 768px, usa esta mediana -->
  <source media="(min-width: 768px)" srcset="banner-tablet.jpg">

  <!-- Imagen por defecto para Mobile o si el navegador no soporta <picture> -->
  
</picture>
```

¿Por qué usarla en tu TP y parcial?

1. **Optimización de Rendimiento:** No descargas una imagen gigante de 2MB en un celular 4G.
2. **Dirección Artística:** Puedes usar una foto con encuadre vertical para celulares y una horizontal apaisada para monitores.
3. **Formatos Modernos:** Puedes ofrecer imágenes WebP o AVIF (mucho más livianas) y dejar un `.png` o `.jpg` tradicional como fallback.

5. JavaScript: Fundamentos Esenciales

JavaScript le da dinamismo e interactividad a nuestras páginas web.

Variables: Declaración y Tipos

En JavaScript moderno declaramos variables de dos formas principalmente:

- `const`: Para valores que **no van a cambiar** de referencia. (¡Úsalo por defecto!).
- `let`: Para valores que **sí van a cambiar** (acumuladores, flags, contadores).
- `var`: **No recomendado** en JS moderno debido a problemas con el alcance (scope) de las variables.

Tipos de Datos Comunes

1. **Booleanos:** `true` o `false`.
2. **Textos (Strings):** `"Hola UADE"`, `'desarrollo web'`, ``Template literals con ${variable}``.
3. **Arrays (Arreglos):** Colecciones ordenadas indexadas desde `0`.

Representación de un Objeto y Array (Ejemplo Messi)

```
// Representando a Messi como un Objeto (para agrupar propiedades de una entidad)
const messi = {
  nombre: "Lionel",
  apellido: "Messi",
  edad: 38,
  estaActivo: true,
  clubes: ["Barcelona", "PSG", "Inter Miami"]
};

// Acceder a datos del objeto
console.log(messi.nombre); // "Lionel"
console.log(messi.clubes[0]); // "Barcelona" (primer elemento del array de clubes)

// Representando un Array de Jugadores (para listas de elementos similares)
const seleccionArgentina = [
  { nombre: "Lionel", apellido: "Messi", posicion: "Delantero" },
  { nombre: "Emiliano", apellido: "Martínez", posicion: "Arquero" },
  { nombre: "Rodrigo", apellido: "De Paul", posicion: "Mediocampista" }
];

// Acceder al apellido del Dibu Martínez
console.log(seleccionArgentina[1].apellido); // "Martínez"
```

6. Formularios HTML y su Captura en JavaScript

Para procesar información en el lado del cliente (JS), es vital construir formularios accesibles y bien estructurados en HTML.

Atributos Clave e Inputs comunes

- **`type="text"`:** Para cadenas de texto generales.
- **`type="number"`:** Restringe la entrada a números. Acepta atributos `min`, `max` y `step`.
- **`type="email"`:** Valida automáticamente que la entrada tenga formato de correo electrónico (`nombre@dominio.com`).
- **`id`:** Identificador único para vincular con JavaScript (`document.getElementById`) y con la etiqueta `<label>`.
- **`name`:** Clave del dato cuando se envía el formulario al servidor.
- **`for (en <label>)`:** Debe coincidir con el `id` del input asociado. Permite que al hacer clic en el texto de la etiqueta, el cursor se enfoque automáticamente en el input.

Estructura de un `<select>` (Desplegable)

```
<label for="select-provincia">Selecciona tu provincia:</label>
<select id="select-provincia" name="provincia" required>
  <option value="" disabled selected>Selecciona una opción...</option>
  <option value="caba">Ciudad de Buenos Aires</option>
  <option value="pba">Provincia de Buenos Aires</option>
  <option value="cordoba">Córdoba</option>
  <option value="santa-fe">Santa Fe</option>
</select>
```

Capturando el valor con JavaScript:

```
const formulario = document.getElementById("mi-formulario");
const selectProvincia = document.getElementById("select-provincia");

formulario.addEventListener("submit", function(evento) {
  evento.preventDefault(); // Evita que la página se recargue

  const provinciaSeleccionada = selectProvincia.value;
  console.log("Provincia elegida:", provinciaSeleccionada);
  // Mostrará "caba", "pba", "cordoba", etc.
});
```

7. Manipulación del DOM: FlexNav (Menú Hamburguesa)

El típico ejercicio de navegación adaptativa (FlexNav) consiste en abrir y cerrar un menú lateral o desplegable al hacer clic en un botón "hamburguesa". Para esto, usamos la API del DOM a través de `classList`.

Métodos de `classList`:

- `classList.add("nombre-clase")`: Añade la clase CSS al elemento (si ya la tiene, no hace nada).
- `classList.remove("nombre-clase")`: Remueve la clase CSS del elemento.
- `classList.toggle("nombre-clase")`: Si el elemento **tiene** la clase, la remueve. Si **no la tiene**, la añade. Es ideal para interruptores tipo "abrir/cerrar".

Código de Ejemplo Práctico

HTML

```
<header>
  <button id="btn-toggle-menu" class="menu-hamburger">≡</button>
  <nav id="navbar" class="nav-links">
    <a href="#">Inicio</a>
    <a href="#">Servicios</a>
    <a href="#">Contacto</a>
  </nav>
</header>
```

CSS

```
/* Por defecto en mobile, escondemos el menú corriéndolo de la pantalla */
.nav-links {
  position: fixed;
  top: 0;
  right: -250px; /* Escondido a la derecha */
  width: 250px;
  height: 100vh;
  background-color: #333;
  transition: right 0.3s ease; /* Transición suave */
}

/* Clase activa que inyectaremos con JS */
.nav-links.active {
  right: 0; /* Entra a la pantalla */
}
```

JavaScript

```
// 1. Guardar los elementos del DOM en variables
const botonMenu = document.getElementById("btn-toggle-menu");
const navbar = document.getElementById("navbar");

// 2. Escuchar el evento clic en el botón
botonMenu.addEventListener("click", function() {
  // 3. Alternar la clase 'active' para abrir y cerrar el menú
  navbar.classList.toggle("active");
});
```

8. Creación de un Modal Dinámico

Un modal es una ventana superpuesta que interrumpe la navegación del usuario para mostrar información o requerir una acción. Se compone de un fondo oscuro semitransparente (overlay) y la caja contenedora del modal en sí.

Estructura HTML recomendada

```
<!-- Botón para abrir el modal -->
<button id="btn-abrir-modal">Ver Detalles</button>

<!-- Contenedor general del Modal (Overlay) -->
<div id="modal-container" class="modal-overlay hidden">
  <div class="modal-content">
```



```
<span id="btn-cerrar-modal" class="close-btn">&times;</span>
<h2>Título del Modal</h2>
<p>Este es el contenido explicativo del modal.</p>
<button id="btn-aceptar">Aceptar</button>
</div>
</div>
```

CSS Esencial

```
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  background-color: rgba(0, 0, 0, 0.5); /* Fondo oscuro semitransparente */
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 2000;
}

/* Clase de utilidad para ocultar */
.hidden {
  display: none !important;
}

.modal-content {
  background-color: white;
  padding: 30px;
  border-radius: 8px;
  position: relative;
  width: 90%;
  max-width: 500px;
}
```

Lógica de Control en JavaScript

```
// Guardar referencias del DOM en variables
const btnAbrir = document.getElementById("btn-abrir-modal");
const btnCerrar = document.getElementById("btn-cerrar-modal");
const btnAceptar = document.getElementById("btn-aceptar");
const modal = document.getElementById("modal-container");

// Función para abrir modal
function abrirModal() {
  modal.classList.remove("hidden");
}
```

```
// Función para cerrar modal
function cerrarModal() {
  modal.classList.add("hidden");
}

// Asignar listeners de eventos
btnAbrir.addEventListener("click", abrirModal);
btnCerrar.addEventListener("click", cerrarModal);
btnAceptar.addEventListener("click", cerrarModal);

// Opcional: Cerrar haciendo clic fuera del modal (en el overlay)
modal.addEventListener("click", function(evento) {
  if (evento.target === modal) {
    cerrarModal();
  }
});
```

9. Lógica del Carrusel (Slide)

Para programar un carrusel que muestre diferentes imágenes al hacer clic en "Siguiente" o "Anterior", necesitamos un **Array** de imágenes, un **Índice actual** y una estructura condicional (**if / else**) para manejar los límites (volver al inicio si pasamos el final, o ir al final si retrocedemos de la primera).

Diagrama de Flujo del Carrusel

```
flowchart TD
  Start([Inicio]) --> Init[Definir array de imágenes e index = 0]
  Init --> Event[Clic en 'Siguiente']
  Event --> Inc[index = index + 1]
  Inc --> CheckEnd{index >= largo del array?}
  CheckEnd -- Sí --> ResetZero[index = 0]
  CheckEnd -- No --> Render[Actualizar atributo 'src' de la imagen en el DOM]
  ResetZero --> Render
  Render --> End([Fin])
```

Pseudocódigo para escribir en el Examen Parcial

```
ALGORITMO Carrusel
VARIABLES:
  imagenes: ARRAY DE CADENAS = ["img1.jpg", "img2.jpg", "img3.jpg"]
  indiceActual: ENTERO = 0
  imgElemento: ELEMENTO_DOM
```

```
PROCEDIMIENTO Siguiente()
    indiceActual <- indiceActual + 1

    SI indiceActual >= longitud(imagenes) ENTONCES
        indiceActual <- 0
    FIN SI

    MOSTRAR_IMAGEN()
FIN PROCEDIMIENTO

PROCEDIMIENTO Anterior()
    indiceActual <- indiceActual - 1

    SI indiceActual < 0 ENTONCES
        indiceActual <- longitud(imagenes) - 1
    FIN SI

    MOSTRAR_IMAGEN()
FIN PROCEDIMIENTO

PROCEDIMIENTO MOSTRAR_IMAGEN()
    imgElemento.src <- imagenes[indiceActual]
FIN PROCEDIMIENTO
```

Código JS real del Carrusel

```
// Array con rutas de las fotos
const imagenes = ["messi1.jpg", "messi2.jpg", "messi3.jpg"];
let indiceActual = 0;

// Referencias de elementos HTML
const imgDisplay = document.getElementById("carrusel-img");
const btnSiguiente = document.getElementById("btn-sig");
const btnAnterior = document.getElementById("btn-ant");

function actualizarImagen() {
    imgDisplay.src = imagenes[indiceActual];
}

btnSiguiente.addEventListener("click", function() {
    indiceActual++;
    // Si llegamos al final del array, volvemos a la primera imagen (index 0)
    if (indiceActual >= imagenes.length) {
        indiceActual = 0;
    }
    actualizarImagen();
});

btnAnterior.addEventListener("click", function() {
    indiceActual--;
```

```
// Si retrocedemos de la primera imagen, vamos a la última (largo - 1)
if (indiceActual < 0) {
  indiceActual = imagenes.length - 1;
}
actualizarImagen();
});
```

10. Calculadora de Impacto

Una "Calculadora de Impacto" es un ejercicio recurrente donde se ingresan valores de consumo (ej. consumo eléctrico, uso de agua, kilómetros recorridos), se multiplican por un factor de impacto (huella de carbono, costo unitario) y se muestra un veredicto o resultado categorizado por niveles (Alto, Medio, Bajo).

Diagrama de Flujo de la Calculadora de Impacto

```
flowchart TD
    Start([Inicio: Envío de Formulario]) --> Prevent[Prevenir recarga de página: event.preventDefault]
    Prevent --> ReadInputs[Leer valor ingresado y factor de impacto]
    ReadInputs --> Validate{¿Es un número válido mayor a 0?}
    Validate -- No --> Error[Mostrar mensaje de error al usuario]
    Validate -- Sí --> Calc[Calcular impacto = consumo * factor]
    Calc --> Compare{¿impacto > 100?}
    Compare -- Sí --> CatHigh[Category: Alto Impacto ●]
    Compare -- No --> Compare2{¿impacto >= 50?}
    Compare2 -- Sí --> CatMed[Category: Medio Impacto ●]
    Compare2 -- No --> CatLow[Category: Bajo Impacto ●]
    CatHigh --> RenderHigh[Mostrar resultado y categoría en pantalla]
    CatMed --> RenderMed[Render]
    CatLow --> RenderLow[Render]
    RenderHigh --> End([Fin])
    RenderMed --> End
    RenderLow --> End
```

Pseudocódigo de la Calculadora de Impacto

```
ALGORITMO CalculadoraImpacto
VARIABLES:
  consumoKWh: REAL
  FACTOR_CO2: REAL = 0.45 // kg de CO2 por cada kWh
  huellaTotal: REAL
  categoria: CADENA
```

```
PROCEDIMIENTO Calcular(evento)
  Llamar a evento.preventDefault() // Evita recargar la página

  consumoKWh <- LEER_VALOR_NUMERICO("input-consumo")

  SI ES_NAN(consumoKWh) O consumoKWh < 0 ENTONCES
    MOSTRAR_MENSAJE("Por favor ingrese un número positivo válido.")
    RETORNAR
  FIN SI

  huellaTotal <- consumoKWh * FACTOR_CO2

  SI huellaTotal > 100 ENTONCES
    categoria <- "Alto Impacto Ambiental (Rojo)"
  SINO SI huellaTotal >= 50 ENTONCES
    categoria <- "Medio Impacto Ambiental (Amarillo)"
  SINO
    categoria <- "Bajo Impacto Ambiental (Verde)"
  FIN SI

  MOSTRAR_RESULTADO(huellaTotal, categoria)
FIN PROCEDIMIENTO
```

Código JS de la Calculadora de Impacto

```
const formularioCalc = document.getElementById("form-calculadora");
const inputConsumo = document.getElementById("consumo-kwh");
const contenedorResultado = document.getElementById("resultado-impacto");

const FACTOR_CO2 = 0.45; // 0.45 kg CO2 por cada kWh consumido

formularioCalc.addEventListener("submit", function(event) {
  event.preventDefault(); // Evitamos recarga

  // Convertimos el valor ingresado a número decimal (float)
  const consumo = parseFloat(inputConsumo.value);

  // Validación de seguridad
  if (isNaN(consumo) || consumo < 0) {
    contenedorResultado.innerHTML = `

Por favor ingresa un consumo válido.</p>`;
    return;
  }

  // Cálculo
  const huellaTotal = consumo * FACTOR_CO2;
  let categoria = "";
  let claseCSS = "";


```

```
// Lógica condicional
if (huellaTotal > 150) {
  categoria = "Alto";
  claseCSS = "impacto-alto"; // Clase CSS para pintar de rojo
} else if (huellaTotal >= 50) {
  categoria = "Medio";
  claseCSS = "impacto-medio"; // Clase CSS para pintar de amarillo
} else {
  categoria = "Bajo";
  claseCSS = "impacto-bajo"; // Clase CSS para pintar de verde
}

// Renderizado en el DOM
contenedorResultado.innerHTML = `
  <div class="card-resultado ${claseCSS}">
    <p>Tu huella es de: <strong>${huellaTotal.toFixed(2)} kg de CO2</strong></p>
    <p>Nivel de impacto: <strong>${categoria}</strong></p>
  </div>
`;
});
```

11. Guía Oficial de Métodos y Propiedades JavaScript (UADE)

Esta sección explica detalladamente y de forma sencilla qué hace cada uno de los métodos y propiedades oficiales utilizados en la materia de Desarrollo Web.

Selección de Elementos en el DOM

1. `document.getElementById("id")`

- **¿Qué hace?** Busca en el HTML el **único** elemento que tenga exactamente el `id` indicado (ej. `id="mi-boton"`) y lo devuelve.
- **Ejemplo:**

```
const boton = document.getElementById("btn-enviar");
```

2. `document.querySelector("selector_css")`

- **¿Qué hace?** Busca el **primer** elemento de la página que coincida con el selector CSS especificado. Sirve para seleccionar por clase (`.clase`), por ID (`#id`) o por etiqueta (`h1`, `div`, `nav` a). Es más genérico y moderno que `getElementById`.
- **Ejemplo:**

```
const primerEnlace = document.querySelector(".nav-links a"); // Trae el primer enlace dentro de la barra de navegación
```

```
const boton = document.querySelector("#btn-enviar"); // Equivale a  
getElementById
```

3. `document.querySelectorAll("selector_css")`

- **¿Qué hace?** Busca **todos** los elementos en la página que coincidan con el selector CSS y los devuelve agrupados en una lista (llamada *NodeList*, que se comporta casi igual que un Array).
- **Ejemplo:**

```
const todosLosEnlaces = document.querySelectorAll(".nav-links a");  
// Ahora 'todosLosEnlaces' es una lista con todos los elementos que cumplen  
la condición.
```

📦 Manipulación de Clases CSS (`classList`)

4. `elemento.classList.add("nombre-clase")`

- **¿Qué hace?** Le añade una clase de CSS al elemento especificado. Esto se usa para cambiar su apariencia dinámicamente inyectándole estilos que ya declaraste en tu CSS (como `.activo`, `.visible`, `.error`).
- **Ejemplo:**

```
tarjeta.classList.add("borde-rojo"); // Le agrega la clase 'borde-rojo' a la  
tarjeta
```

5. `elemento.classList.remove("nombre-clase")`

- **¿Qué hace?** Quita la clase CSS indicada del elemento. Si el elemento no tiene la clase, no hace nada.
- **Ejemplo:**

```
tarjeta.classList.remove("borde-rojo"); // Quita la clase y vuelve a su  
estado normal
```

6. `elemento.classList.toggle("nombre-clase")`

- **¿Qué hace?** Funciona como un interruptor de luz. Si el elemento **ya tiene** esa clase, se la remueve; si **no la tiene**, se la agrega. Es ideal para abrir/cerrar menús colapsables o cambiar entre modo oscuro y claro.
- **Ejemplo:**

```
menu.classList.toggle("active"); // Si está abierto lo cierra, si está  
cerrado lo abre
```

🔊 Escuchadores de Eventos

7. `elemento.addEventListener("evento", funcion)`

- **¿Qué hace?** Queda "escuchando" a que ocurra una acción del usuario en ese elemento (como un "click", "submit" de formulario, o "keydown" de teclado) para ejecutar la función asociada inmediatamente.
- **Ejemplo:**

```
boton.addEventListener("click", function() {  
    alert("¡Clic hecho!");  
});
```

🔧 Creación, Inserción y Eliminación de Elementos

8. `document.createElement("etiqueta")`

- **¿Qué hace?** Crea un nuevo elemento HTML en la memoria de JavaScript (todavía no se muestra en la pantalla). Tienes que indicarle qué etiqueta quieres crear (ej. "p", "div", "li", "img").
- **Ejemplo:**

```
const nuevoParrafo = document.createElement("p"); // Crea un elemento <p>  
vacío en memoria  
nuevoParrafo.innerHTML = "Hola, soy nuevo!";
```

9. `document.body.appendChild(elemento) / padre.appendChild(elemento)`

- **¿Qué hace?** Inserta el elemento que creaste (en el paso anterior) dentro de otro elemento en la página. `appendChild` lo mete al final, como el "último hijo".
 - `document.body.appendChild(parrafo)` lo mete al final de todo el body.
 - `contenedor.appendChild(parrafo)` lo mete adentro de ese contenedor específico.
- **Ejemplo:**

```
const contenedor = document.getElementById("contenedor-mensajes");  
contenedor.appendChild(nuevoParrafo); // Ahora el párrafo aparece  
físicamente en la web
```


10. `elemento.remove()`

- **¿Qué hace?** Elimina por completo ese elemento HTML de la página web.
- **Ejemplo:**

```
const anuncio = document.getElementById("anuncio-publicitario");
anuncio.remove(); // Desaparece físicamente del HTML de la pantalla
```

Modificación de Contenido y Propiedades

11. `elemento.innerHTML`

- **¿Qué hace?** Es una propiedad que representa el contenido HTML que hay dentro de una etiqueta. Si le asignas un valor con el signo `=`, sobrescribe todo su contenido. Puedes usarlo para insertar texto simple o texto con etiquetas HTML.
- **Ejemplo:**

```
caja.innerHTML = "<strong>Texto en negrita</strong>";
```

Bucles y Recorridos de Listas

12. `lista.forEach(funcion)`

- **¿Qué hace?** Sirve para recorrer (iterar) uno por uno todos los elementos de un array o de una lista obtenida con `querySelectorAll()`. Ejecuta la función que le pases para cada uno de los elementos.
- **Ejemplo:** Supongamos que queremos pintar de rojo todos los enlaces de la página:

```
const todosLosEnlaces = document.querySelectorAll("a");

// Para cada 'enlace' de la lista de enlaces, agregale la clase 'rojo'
todosLosEnlaces.forEach(function(enlace) {
  enlace.classList.add("rojo");
});
```